RESERVA DE SALAS

1. CONTEXTO

Un centro educativo necesita una aplicación web para gestionar las salas disponibles y las reservas realizadas por sus usuarios.

La aplicación dispone de una interfaz web desde la que se pueden consultar las salas, comprobar su disponibilidad para una fecha y una franja horaria determinadas y gestionar las reservas. También será necesario controlar qué salas están activas y evitar que una misma sala pueda tener varias reservas confirmadas cuyos horarios se solapen.

La aplicación se desarrollará utilizando **PHP y MySQL**, siguiendo el patrón **MVC** y separando el acceso a datos de la lógica de negocio y de la presentación.

La interfaz web y el código JavaScript necesarios para realizar las peticiones a la API se proporcionan preparados. El alumnado deberá implementar la parte del servidor en PHP.

2. OBJETIVOS

- Desarrollar una API REST con PHP.
- Conectar PHP con una base de datos MySQL.
- Procesar peticiones HTTP y devolver respuestas en formato JSON.
- Consultar salas y comprobar su disponibilidad.
- Implementar las operaciones de consulta y gestión de reservas.
- Validar los datos recibidos antes de almacenarlos.
- Evitar reservas duplicadas o incompatibles por solapamiento de horarios.
- Utilizar consultas preparadas para acceder a la base de datos.
- Gestionar errores y devolver códigos de estado HTTP adecuados.
- Separar el código mediante modelos, DAO, controladores y puntos de entrada de la API.
- Comprobar el funcionamiento de la API utilizando la interfaz proporcionada.

2. OPERACIONES QUE DEBE IMPLEMENTAR LA APLICACIÓN

Tabla salas

Consultar salas

- Mostrar las salas registradas en la base de datos.
- Permitir filtrar las salas para mostrar únicamente las activas.
- Permitir filtrar por tipo de sala.
- Permitir filtrar por capacidad mínima.
- Comprobar la disponibilidad de las salas para una fecha y una franja horaria determinadas.

La respuesta deberá incluir los datos de cada sala y un indicador que permita conocer si está disponible para el intervalo solicitado.

Tabla reservas

Consultar y gestionar reservas

- Consultar las reservas registradas.
- Registrar una nueva reserva.
- Consultar una reserva a partir de su identificador, si la interfaz proporcionada lo requiere.
- Modificar el estado de una reserva cuando la operación esté contemplada en la interfaz y en el contrato de la API.
- Comprobar que la sala seleccionada existe y está activa.
- Evitar que se confirme una reserva cuando exista otra reserva confirmada para la misma sala y con un horario incompatible.

Restricción: La gestión de las reservas deberá respetar las restricciones definidas en la base de datos y las operaciones que utilice la interfaz proporcionada.

Importante: no se deben implementar operaciones que no estén contempladas en el contrato de la API o que no sean necesarias para las funcionalidades solicitadas.

3. BASE DE DATOS

La base de datos utilizada será **reservas_espacios.**

Trabajareis con el esquema de base de datos proporcionado para el ejercicio. No será necesario crear una base de datos nueva ni modificar su estructura, salvo que se indique expresamente.

La base de datos contiene, entre otras, las siguientes tablas:

- **salas**: almacena la información de las salas, incluyendo su identificador, nombre, ubicación, capacidad, tipo y estado de actividad.
- **reservas**: almacena las reservas realizadas, incluyendo la sala reservada, la fecha, la hora de inicio, la hora de fin y el estado de la reserva.

Las reservas están relacionadas con las salas mediante sus identificadores. Se deberá respetar la integridad referencial definida en la base de datos. La conexión a MySQL deberá centralizarse en la clase correspondiente de acceso a datos o configuración, evitando repetir los parámetros de conexión en distintos archivos.

4. FUNCIONALIDADES

Tabla salas: Consulta

La API deberá permitir obtener el listado de salas con la siguiente información:

- Identificador de la sala.
- Nombre.
- Ubicación.
- Capacidad.
- Tipo.
- Estado de actividad.
- Disponibilidad para la fecha y franja horaria solicitadas.

La consulta deberá admitir los filtros que utiliza la interfaz:

- **activas:** permite filtrar las salas activas.
- **fecha:** fecha para la que se quiere comprobar la disponibilidad.
- **hora_inicio**: hora de inicio del intervalo solicitado.
- **hora_fin:** hora de finalización del intervalo solicitado.

También se deberán admitir los filtros de tipo y capacidad mínima cuando estén incluidos en la petición.

Una sala no estará disponible si está inactiva o si existe una reserva confirmada que se solapa con el intervalo solicitado.

Para detectar el solapamiento, se deberá tener en cuenta que una reserva existente se solapa con la nueva solicitud cuando su hora de inicio es anterior a la hora de finalización solicitada y su hora de fin es posterior a la hora de inicio solicitada.

Por ejemplo, si una sala tiene una reserva confirmada de 09:00 a 10:00:

- Una solicitud de 08:00 a 09:00 no se solapa.
- Una solicitud de 10:00 a 11:00 no se solapa.
- Una solicitud de 09:30 a 10:30 sí se solapa.

Tabla reservas: consultar, crear y actualizar

La API deberá implementar las operaciones necesarias para gestionar las reservas desde la interfaz proporcionada.

Consultar reservas

- Obtener las reservas almacenadas.
- Devolver los datos necesarios para mostrarlas en la interfaz.
- Permitir consultar una reserva concreta si así lo requiere el contrato de la API.

Crear una reserva

- Recibir los datos de la reserva.
- Validar que los campos obligatorios estén informados.
- Comprobar que la sala existe y está activa.
- Validar la fecha y las horas recibidas.
- Comprobar que la hora de inicio sea anterior a la hora de fin.
- Comprobar que no exista otra reserva confirmada que se solape con el intervalo solicitado.
- Registrar la reserva si todas las comprobaciones son correctas.

Actualizar una reserva

Si la interfaz contempla esta operación, la API deberá validar los datos modificados y volver a comprobar la disponibilidad de la sala. Al realizar la comprobación, no se deberá considerar la propia reserva como un conflicto consigo misma.

Cambiar el estado de una reserva

Si la interfaz contempla el cambio de estado, la API deberá comprobar que la reserva existe y que el estado solicitado es válido. La operación deberá respetar las reglas definidas para las reservas.

Después de cada operación correcta, la interfaz deberá poder actualizar la información mostrada.

5. CONTRATO DE LA API

La API se dividirá en dos puntos de entrada:

- **/api/salas.php**
- **/api/reservas.php**

API de salas: /api/salas.php

GET

Parametro: activas

Funcionalidad: Filtrar las salas activas cuando su valor sea 1.

Parametro: fecha

Funcionalidad: Fecha para comprobar la disponibilidad.

Parametro: hora_inicio

Funcionalidad: Hora inicial del intervalo solicitado.

Parametro: hora_fin

Funcionalidad: Hora final del intervalo solicitado.

Parametro: tipo

Funcionalidad: Filtrar por tipo de sala, si se proporciona.

Parametro: capacidad_minim

Funcionalidad: Filtrar por capacidad mínima, si se proporciona.

API de reservas: /api/resservas.php

GET

Parametro: Ninguno.

Funcionalidad: Consultar las reservas.

POST

Parametro: JSON en el cuerpo.

Funcionalidad: Registrar una nueva reserva.

PUT

Parametro: ?id=1 y JSON en el cuerpo

Funcionalidad: Modificar una reserva existente, si está contemplado en la interfaz.

PATCH

Parametro: ?id=1 y JSON en el cuerpo

Funcionalidad: Modificar parcialmente una reserva, incluido su estado, si está contemplado en la interfaz.

Respuestas HTTP

Todas las respuestas de la API deberán estar en formato JSON, tanto en las operaciones correctas como en los errores.

Se utilizarán códigos HTTP apropiados, entre ellos:

- 200 OK: operación realizada correctamente.
- 201 Created: reserva creada correctamente.
- 400 Bad Request: datos incorrectos o petición no válida.
- 404 Not Found: sala o reserva inexistente.
- 405 Method Not Allowed: método HTTP no permitido.
- 409 Conflict: conflicto de disponibilidad al intentar realizar una reserva.
- 500 Internal Server Error: error interno del servidor.

La API deberá rechazar los métodos HTTP que no estén permitidos para cada recurso

6. VALIDACIONES

Validaciones de salas

- El identificador recibido, cuando corresponda, deberá ser válido.
- Los filtros de capacidad deberán tener valores numéricos válidos.
- El tipo de sala deberá respetar los valores admitidos por la aplicación.
- Las salas inactivas no podrán utilizarse para realizar nuevas reservas.
- Si se solicita comprobar la disponibilidad, la fecha y las horas deberán tener un formato válido.

Validaciones de reservas

- Los campos obligatorios deberán estar informados.
- La sala indicada deberá existir en la base de datos.
- La sala deberá estar activa.
- La fecha deberá tener un formato válido.
- Las horas deberán tener un formato válido.
- La hora de inicio deberá ser anterior a la hora de fin.
- No se permitirá confirmar una reserva que se solape con otra reserva confirmada de la misma sala.
- El identificador deberá corresponder a una reserva existente cuando se realice una consulta o modificación concreta.
- Al modificar una reserva, se deberá comprobar de nuevo la disponibilidad.
- Los estados recibidos deberán pertenecer a los valores admitidos por la aplicación.

Validaciones generales

- Utilizar consultas preparadas para evitar inyecciones SQL.
- Validar los datos recibidos antes de ejecutar operaciones de escritura.
- Devolver respuestas JSON tanto en las operaciones correctas como en los errores.
- Utilizar códigos HTTP adecuados para identificar los errores.
- Gestionar los errores de conexión y ejecución de consultas sin mostrar información interna de la base de datos al usuario.
- No permitir operaciones que no estén contempladas en el contrato de cada API.
- Mantener la separación entre acceso a datos, lógica de negocio y presentación.

